Grammar School “Jovan Jovanovic Zmaj”

Novi Sad, Republic of Serbia

Matura Paper in Computer Science and Informatics

Convex Polyhedrons in Computer Graphics

Proffesor Mentor:

Dragan Masulovic, Ph.D.

Student:

Luka Lehotski

Novi Sad, May 2023.

Preface

Mathematics was my favorite subject in elementary school. I found it fascinating to solve problems, especially those in geometry. Geometry, and particularly solid geometry, has always been interesting to me because it is visual and is all around us. The most enjoyable part of solving problems was drawing geometric shapes, and further observations and calculations were not difficult when I had a good image from which I could discern patterns. Although mathematics still interests me, my focus shifted to computer science in high school. I was particularly drawn to the field of computer graphics because it represents a combination of computer science and geometry, so I decided to write my graduation thesis in this area—a program for drawing convex polyhedrons in the C# programming language. This program does exactly what I found most interesting in solving geometry problems, and at the same time, it serves as a foundation for a more serious graphic application.

I would like to thank my parents first and foremost for their support throughout my education and the writing of this thesis, as well as Professor Mašulović for sparking my interest in computer science, introducing me to this field, and mentoring me in this work.

Table of Contents

1 Introduction

The visual representation of geometric objects is very important in robotics and other branches of industry. Computer graphics deals with this, and in this thesis, which focuses on it, the drawing of convex polyhedrons as a group of geometric bodies that are simple to define will be presented. We limit ourselves to convex polyhedrons because certain procedures (hiding edges) are very complex for non-convex polyhedrons. In solving the problem of drawing convex polyhedrons, the following questions arise:

- How to represent a convex polyhedron in memory and save data about it?
- How is a convex polyhedron (three dimensions) represented on the screen (two dimensions)?
- How to recognize which edges of the polyhedron should be visible and which should be hidden?
- How to implement elementary transformations with the obtained polyhedron?

These questions represent both the chapters of this thesis and the process of my thinking about this problem, which led to the final solution.

2 Representation of a Convex Polyhedron in Memory

It is necessary to create data structures in memory that will store all the information about the polyhedron. Since every polyhedron consists of vertices, edges, and faces, we will create separate data structures for each of these.

2.1. Data Structures for Storing a Convex Polyhedron

Vertices are points in space, so the structure for a vertex, Vertex, will have fields x, y, z, which represent the coordinates (as decimal numbers) of that point in space.

```
class Vertex {
    public float x, y, z;
    public Vertex(float x_coord, float y_coord, float z_coord) {
        x = x_coord;
        y = y_coord;
        z = z_coord;
    }
}
```

Edges consist of two vertices, so we will store an edge in the Edge structure as a pair of integers that are the indices of the vertices in that edge.

```
class Edge {
    public int ind1, ind2;
    public Edge(int i1, int i2) {
        ind1 = i1;
        ind2 = i2;
    }
}
```

The faces of the polyhedron will be represented in the Face structure as an array with the indices of the vertices in that face.

```
class Face {
    public int[] points;
    public Face(int[] face_vertices) {
        points = new int[face_vertices.Length];
        for (int i = 0; i < points.Length; i++) {
            points[i] = face_vertices[i];
        }
    }
}
```

Now that we have these structures, we can create a structure for the entire polyhedron, which will contain information about its vertices, edges, and faces. The Polyhedron structure consists of three arrays: an array of vertices, an array of edges, and an array of faces.

```
class Polyhedron {
    public Vertex[] vertices;
    public Edge[] edges;
    public Face[] faces;

    public Polyhedron(Vertex[] v, Edge[] e, Face[] f) {
        vertices = new Vertex[v.Length];
        edges = new Edge[e.Length];
        faces = new Face[f.Length];
        for (int i = 0; i < v.Length; i++) {
            vertices[i] = v[i];
        }
        for (int i = 0; i < e.Length; i++) {
            edges[i] = e[i];
        }
        for (int i = 0; i < f.Length; i++) {
            faces[i] = f[i];
        }
    }
}
```

2.2. Loading from a Text File

The text file from which we load data about the polyhedron has the following structure:

- Number of vertices of the polyhedron (natural number **nv**)
- **nv** rows where exactly 3 decimal numbers separated by spaces are entered (vertex coordinates)
- Number of edges of the polyhedron (natural number **ne**)
- **ne** rows where exactly 2 natural numbers less than **nv** separated by spaces are entered (indices of the vertices of that edge)
- Number of faces of the polyhedron (natural number **nf**)
- **nf** rows where natural numbers less than **nv** separated by spaces are entered (indices of the vertices of that face)

The program does not perform input control, so it is necessary that the text file be in the previously specified format. Also, if there is no file named "ulaz.txt" in the same directory as the program, the program will print that the text file does not exist.

```
string fname = "ulaz.txt";
Vertex[] vertices = new Vertex[10];
Edge[] edges = new Edge[10];
Face[] faces = new Face[10]; // initialization
int nv = 0, ne = 0, nf = 0, br = 0;
if (File.Exists(fname)) {
    StreamReader input = new StreamReader(fname);
    string s; // used for reading the file
    while (!input.EndOfStream) {
        s = input.ReadLine();
        if (br == 0) {
            nv = int.Parse(s); br++;
            vertices = new Vertex[nv];
            for (int i = 0; i < nv; i++) {
                s = input.ReadLine();
                float[] a = new float[3];
                string[] ss = s.Split(' ');
                for (int j = 0; j < 3; j++) {
                    a[j] = float.Parse(ss[j]);
                }
                vertices[i] = new Vertex(a[0], a[1], a[2]);
                br++;
            }
        }
        else if (br == nv + 1) {
            ne = int.Parse(s); br++;
            edges = new Edge[ne];
            for (int i = 0; i < ne; i++) {
                s = input.ReadLine();
                string[] ss = s.Split(' ');
                edges[i] = new Edge(int.Parse(ss[0]), int.Parse(ss[1]));
                br++;
            }
        }
        else if (br == nv + ne + 2) {
            nf = int.Parse(s); br++;
            faces = new Face[nf];
            for (int i = 0; i < nf; i++) {
                s = input.ReadLine();
                string[] ss = s.Split(' ');
                int[] b = new int[ss.Length];
                for (int j = 0; j < ss.Length; j++) {
                    b[j] = int.Parse(ss[j]);
                }
                faces[i] = new Face(b);
                br++;
            }
        }
    }
    input.Close();
}
```

3 Projection from Three Dimensions to Two Dimensions

The process by which a geometric body (in this case, a convex polyhedron) is represented in a plane is called projection. Each point in space (3 dimensions) that belongs to the polyhedron is mapped to a point in the plane (2 dimensions) using a specific formula. The polyhedron is projected by projecting each of its vertices, and the data about the edges and faces do not change, as the structures for edges and faces store only indices (ordinal numbers of points) that do not change during projection.

There are several types of projections, and in this thesis, two types of projections will be presented, with the goal of displaying the polyhedron at a certain angle from above, i.e., with a view of the *xy-plane* (in other words, to see it from a bird's-eye view).

3.1. Orthographic Projection

Orthographic projection projects the point *(x,y,z)* to the point *(x,y,0)*. In other words, it removes the z-coordinate of each vertex. This loses information about the third dimension of the polyhedron, but orthographic projection is certainly significant for a simplified representation of the polyhedron.

Below is the NormalProjection method for orthographic projection of the polyhedron. It is located within the Polyhedron structure, and by calling *Polyhedron R = Polyhedron.NormalProjection(P)* within the Main method, the polyhedron R, which represents the orthographic projection of the polyhedron P, is obtained.

```
public static Polyhedron NormalProjection(Polyhedron P) {
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        R.vertices[i].z = 0;
    }
    return R;
}
```

3.2. Projection at an Angle of *arctg4*

This projection projects the z-axis onto the line y = x, so that the positive part of the z-axis is in the third quadrant. The point (x,y,z) is projected to the point (x',y') as follows:

$$x' = x - z\frac{\sqrt{2}}{8} \ , \ y' = y - z\frac{\sqrt{2}}{8}.$$

```
public static Polyhedron AngleProj(Polyhedron P) {
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        float x = R.vertices[i].x;
        float y = R.vertices[i].y;
        float z = R.vertices[i].z;
        float k = 0.177f; // sqrt(2)/8
        R.vertices[i].x = x - z * k;
        R.vertices[i].y = y - z * k;
        R.vertices[i].z = 0;
    }
    return R;
}
```

4 Displaying the Polyhedron on a Computer Screen

Now that we can store data about the polyhedron and know how projections work, we need to familiarize ourselves with the drawing methods in the C# programming language to visualize it.

4.1. Drawing in the C# Programming Language

The computer screen represents a rectangular matrix of points. The points are called pixels, and each pixel has its own non-negative integer coordinates, with the coordinate origin located in the upper left corner of the screen.

To start drawing, it is necessary to open a Windows form. Below is the code that launches the form and draws a line with coordinates of the starting and ending points (100,50) and (400,200), as well as the image that the code draws.

```
using System;
using System.Windows.Forms;
using System.Drawing;

class Program {
    static void Main() {
        var frm = new Form();
        frm.Text = "Example of Graphics in C#";
        frm.Size = new Size(600, 600);
        frm.Paint += MyDraw;
        frm.ShowDialog();
    }
    static void MyDraw(object sender, PaintEventArgs e) {
        var g = e.Graphics;
        var brush = new Pen(Color.Red);
        brush.Width = 5;
        g.DrawLine(brush, 100, 50, 400, 200);
    }
}
```

4.2 Adaptation for Conventional Drawing

In the previous image, we can see that the coordinate origin is located in the upper left corner, that the positive part of the x-axis is directed to the right, the positive part of the y-axis is directed downward, and that there are only positive coordinates. When examining the figure, it is much more convenient for the coordinate origin to be in the center so that we have negative coordinates as well, and for the positive part of the y-axis to be directed upward. This is done by the DrawingAdaptation method, which changes the *x* and *y* coordinates of each vertex of the projected polyhedron as follows:

- The drawing will be 800 x 800 pixels in size, so the center is at the point (400,400)
- 400 pixels are added to the x coordinate
- The sign of the y coordinate is changed, and 400 pixels are added

This way, the coordinate origin moves from (0,0) to (400,400), and a point with negative coordinates (-a,-b), (a,b<400) moves to (-a+400,b+400) and is located in the lower left quadrant of the screen (fourth quadrant) where it should otherwise be located on a paper drawing. Below is the code.

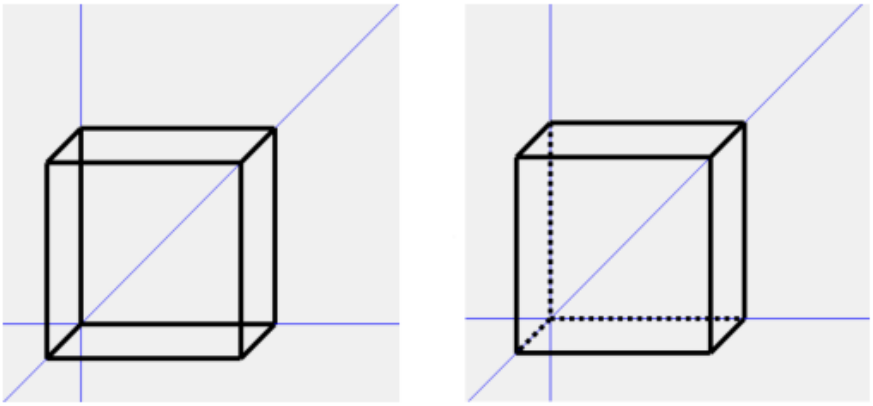
```
public static Polyhedron DrawingAdaptation(Polyhedron P) {
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        R.vertices[i].x = R.vertices[i].x + 400;
        R.vertices[i].y = -R.vertices[i].y + 400;
    }
    return R;
}
```

4.3 Displaying the Polyhedron

Now that we have methods for projection (we can display a three-dimensional object on the screen), a method for correctly orienting the coordinate axes, and know how to draw lines, we can draw the polyhedron by drawing all its edges.

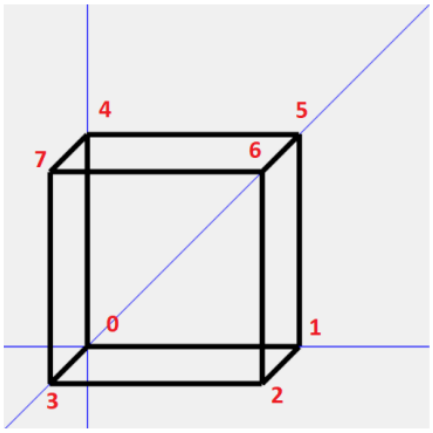
```
// load polyhedron P
Polyhedron pr = Polyhedron.AngleProj(P);
Polyhedron drw = Polyhedron.DrawingAdaptation(pr);
var g = e.Graphics;
var pen = new Pen(Color.Black);
var axis_pen = new Pen(Color.Blue);
pen.Width = 5;
axis_pen.Width = 1;
g.DrawLine(axis_pen, 400, 0, 400, 800);
g.DrawLine(axis_pen, 0, 400, 800, 400);
g.DrawLine(axis_pen, 0, 800, 800, 0);
for (int i = 0; i < pr.edges.Length; i++) {
    float x1 = drw.vertices[pr.edges[i].ind1].x;
    float y1 = drw.vertices[pr.edges[i].ind1].y;
    float x2 = drw.vertices[pr.edges[i].ind2].x;
    float y2 = drw.vertices[pr.edges[i].ind2].y;
    g.DrawLine(pen, x1, y1, x2, y2);
}
```

5 Hiding Invisible Edges of a Convex Polyhedron



To have a realistic representation of the polyhedron, it is necessary to hide its edges that are not visible from the observer's angle, i.e., to draw them with a dashed line. In the image above, we see on the left a cube that the program draws if it simply draws all the edges in order without considering which edge should be hidden (the code written so far), and on the right is the image of how the cube should look (the image we will get after implementing the algorithm described in this chapter).

The algorithm used to remove invisible edges is called the *Backface Culling* algorithm. For the algorithm to work, it is necessary that all its faces be positively oriented. This means that when entering the faces of the polyhedron, we must pay attention to the orientation of each face: the order of the vertex indices in each face must be numbered in the counterclockwise direction (positive mathematical direction) when looking at that face from the front. Below is an illustration of the correct numbering of faces:



1. Each face of the convex polyhedron is projected into a convex polygon. We observe each of the resulting polygons. If the polygon is positively oriented, the corresponding face of the polyhedron is visible. We determine the visibility of each face.

2. An edge is visible if at least one of the faces containing that edge is visible. Below are additional explanations and codes for both parts of the algorithm.

5.1 Determining Face Visibility

We determine the visibility of a face by observing three consecutive vertices of that face. If the vertex with index 2 is to the left of the line determined by the vertices with indices 0 and 1, that face is positively oriented. To determine whether point P is to the left or right of the line determined by points A and B, we use the formula:

$$\lambda = (y_B - y_A) \cdot x_P - (x_B - x_A) \cdot y_P + y_A \cdot x_B - x_A \cdot y_B$$

If λ > 1, point P is to the left of the line, so the polygon is positively oriented, and if λ < 1, it is to the right and negatively oriented. The method below, *VisibleOrInvisible*, gives us an array that tells us which faces of the polyhedron are visible and which are not: *VisibleOrInvisible[i]=true* if the i-th face of the polyhedron is visible (positively oriented).

```
static bool[] VisibleOrInvisible(Polyhedron P) {
    int n = P.faces.Length;
    bool[] v = new bool[n];
    for (int i = 0; i < n; i++) {
        v[i] = true; } // initialization
    for (int i = 0; i < n; i++) {
        float xA = P.vertices[P.faces[i].points[0]].x;
        float yA = P.vertices[P.faces[i].points[0]].y;
        float xB = P.vertices[P.faces[i].points[1]].x;
        float yB = P.vertices[P.faces[i].points[1]].y;
        float xp = P.vertices[P.faces[i].points[2]].x;
        float yp = P.vertices[P.faces[i].points[2]].y;
        float
```

5.2 Determining the Visibility of Edges

It is necessary to go through all the edges of the polyhedron and determine which two faces each edge belongs to. The method Contains checks whether the *i*-th edge of the polyhedron is part of the *j*-th face (it checks if both vertices of the edge are in that face), and if so, it returns true. If the *j*-th face contains the *i*-th edge and the *j*-th face is visible, that is, if Contains(P, i, j) = true and visible[j] = true, then the edge is visible (drawn with a solid line), so full_edge[i] = true.

```
bool[] visible = VisibleOrInvisible(drw);
bool[] full_edge = new bool[drw.edges.Length];
for (int i = 0; i < drw.edges.Length; i++) {
    bool t = false;
    for (int j = 0; j < drw.faces.Length; j++) {
        if (Contains(drw, i, j)) {
            t = t || visible[j];
        }
    }
    full_edge[i] = t;
}

static bool Contains(Polyhedron P, int i, int j) {
    int i1 = P.edges[i].ind1;
    int i2 = P.edges[i].ind2;
    int b = 0;
    for (int k = 0; k < P.faces[j].points.Length; k++) {
        if (i1 == P.faces[j].points[k] || i2 == P.faces[j].points[k]) {
            b++;
        }
    }
    return b == 2;
}
```

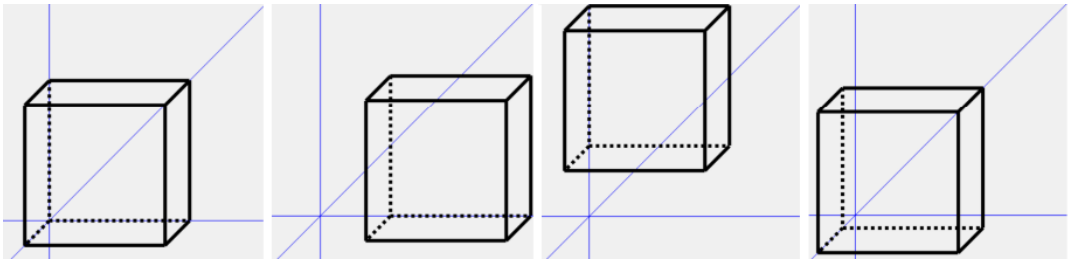
5.3 Drawing Hidden Edges

It is necessary to modify the part of the code related to drawing the polyhedron (Chapter 4.3). The line is made dashed by setting the DashPattern property to the dashValues array. When we iterate through the edges, if visible[i] = false, the *i*-th edge is dashed.

```
var g = e.Graphics;
var pen = new Pen(Color.Black);
var dashed_pen = new Pen(Color.Black);
var axis_pen = new Pen(Color.Blue);
pen.Width = 5;
dashed_pen.Width = 5;
axis_pen.Width = 1;
g.DrawLine(axis_pen, 400, 0, 400, 800);
g.DrawLine(axis_pen, 0, 400, 800, 400);
g.DrawLine(axis_pen, 0, 800, 800, 0);
float[] dashValues = {1, 1, 1, 1};
dashed_pen.DashPattern = dashValues;
for (int i = 0; i < pr.edges.Length; i++) {
    float x1 = drw.vertices[pr.edges[i].ind1].x;
    float y1 = drw.vertices[pr.edges[i].ind1].y;
    float x2 = drw.vertices[pr.edges[i].ind2].x;
    float y2 = drw.vertices[pr.edges[i].ind2].y;
    if (full_edge[i]) {
        g.DrawLine(pen, x1, y1, x2, y2);
    } else {
        g.DrawLine(dashed_pen, x1, y1, x2, y2);
    }
}
```

6 Elementary Transformations of Convex Polyhedrons

Transformations of polyhedrons are very important for viewing them in different ways. It is useful to have an enlarged view, a view from a different perspective, and a view of a shifted polyhedron, and transformations and their compositions allow us to do just that.

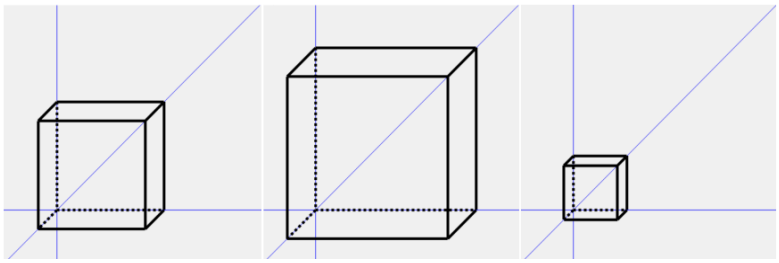


6.1 Translation

In the image above, we see, in order, a normal cube, a cube translated along the x-axis, the y-axis, and the z-axis. Translation is an isometric transformation that maps a polyhedron P to a polyhedron R by a vector v as follows: each vertex (x,y,z) of the polyhedron is transformed into a vertex $(x+a,y+b,z+c)$ where a,b, and c are the components of the vector v. If we want to translate the polyhedron along the y-axis by 100 pixels, for example, then $v=(0,100,0)$. Below is the method for translating a polyhedron, which is called with Polyhedron R = Polyhedron.Translate(P, a, b, c).

```
public static Polyhedron Translate(Polyhedron P, float a, float b, float c) {
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        R.vertices[i].x = R.vertices[i].x + a;
        R.vertices[i].y = R.vertices[i].y + b;
        R.vertices[i].z = R.vertices[i].z + c;
    }
    return R;
}
```

6.2 Scaling



To have a better view of the polyhedron, it is useful to have methods for enlarging and reducing the polyhedron, i.e., changing its size. The Scale method works by calling Polyhedron R = Polyhedron.Scale(P, sc), which creates a new polyhedron where all edge lengths are multiplied by sc (if sc > 1, the polyhedron is enlarged; if sc < 1, it is reduced). In other words, each vertex is mapped from x,y,z) to $sc*x,sc*y,sc*z$).

```
public static Polyhedron Scale(Polyhedron P, float sc) {
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        R.vertices[i].x = R.vertices[i].x * sc;
        R.vertices[i].y = R.vertices[i].y * sc;
        R.vertices[i].z = R.vertices[i].z * sc;
    }
    return R;
}
```

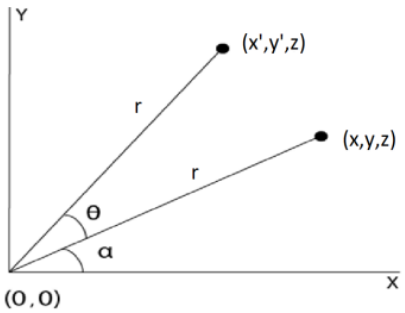
6.3 Rotation

Rotation is a transformation that is useful for viewing the polyhedron from different angles. Of these three transformations, rotation is the most complex, and its derivation is not straightforward, so we will not derive it here. Below, we present the derivation of the formula for rotation around the z-axis, and the derivations for rotation around the x and y axes are done similarly. In rotation around the z-axis, the z-coordinate of each vertex remains the same, so we only consider how the x and y coordinates change. We use the normal projection (see Chapter 3.1). The distance from the origin (r)(r) of each vertex does not change.

$$x = r * \cos\alpha, y = r * \sin\alpha, z = z$$

$$x' = r * \cos(\alpha + \theta) = r * (\cos\alpha * \cos\theta - \sin\alpha * \sin\theta)$$
$$y' = r * \sin(\alpha + \theta) = r * (\sin\alpha * \cos\theta + \cos\alpha * \sin\theta)$$
$$z' = z$$

The rotation matrices (see image on the right) around the x, y, and z axes represent another way to write the formulas for coordinate transformations. To derive rotation around an arbitrary axis, we need to compose three rotations, which requires a lot of calculation. In the next chapter, a faster method for rotating a polyhedron around any axis is presented.



$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi & \cos\varphi \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos\varphi & 0 & \sin\varphi \\ 0 & 1 & 0 \\ -\sin\varphi & 0 & \cos\varphi \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

```
public static Polyhedron RotateX(Polyhedron P, float a) {
    double b = (double)a * ((Math.PI) / 180);
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        float X = R.vertices[i].x;
        float Y = R.vertices[i].y;
        float Z = R.vertices[i].z;
        R.vertices[i].x = X;
        R.vertices[i].y = Y * (float)Math.Cos(b) - Z * (float)Math.Sin(b);
        R.vertices[i].z = Y * (float)Math.Sin(b) + Z * (float)Math.Cos(b);
    }
    return R;
}

public static Polyhedron RotateY(Polyhedron P, float a) {
    double b = (double)a * ((Math.PI) / 180);
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        float X = R.vertices[i].x;
        float Y = R.vertices[i].y;
        float Z = R.vertices[i].z;
        R.vertices[i].x = X * (float)Math.Cos(b) + Z * (float)Math.Sin(b);
        R.vertices[i].y = Y;
        R.vertices[i].z = -X * (float)Math.Sin(b) + Z * (float)Math.Cos(b);
    }
    return R;
}

public static Polyhedron RotateZ(Polyhedron P, float a) {
    double b = (double)a * ((Math.PI) / 180);
    Polyhedron R = Polyhedron.Copy(P);
    for (int i = 0; i < R.vertices.Length; i++) {
        float X = R.vertices[i].x;
        float Y = R.vertices[i].y;
        float Z = R.vertices[i].z;
        R.vertices[i].x = X * (float)Math.Cos(b) - Y * (float)Math.Sin(b);
        R.vertices[i].y = X * (float)Math.Sin(b) + Y * (float)Math.Cos(b);
        R.vertices[i].z = Z;
    }
    return R;
}
```

6.3.1 Rotation Using Quaternions

Quaternions represent an extension of the set of complex numbers. They have four dimensions: one real and three imaginary: i,j, and k. They hold a very important place in geometry, primarily because they describe any rotation in three dimensions before vectors and analytic geometry as we know them today were developed. They are used in robotics to describe every movement because calculations with them are much faster than with ordinary vectors.

$$q = a + bi + cj + dk$$

$$i^2 = j^2 = k^2 = ijk = -1$$

When talking about rotational quaternions (which are important for rotating polyhedrons), the following holds:

$$\sqrt{a^2 + b^2 + c^2 + d^2} = 1$$

The product t of quaternions r and s is defined as follows:

$$(t_0, t_1, t_2, t_3) = (r_0, r_1, r_2, r_3) \times (s_0, s_1, s_2, s_3)$$

$$t_0 = (r_0 s_0 - r_1 s_1 - r_2 s_2 - r_3 s_3)$$

$$t_1 = (r_0 s_1 + r_1 s_0 - r_2 s_3 + r_3 s_2)$$

$$t_2 = (r_0 s_2 + r_1 s_3 + r_2 s_0 - r_3 s_1)$$

$$t_3 = (r_0 s_3 - r_1 s_2 + r_2 s_1 + r_3 s_0)$$

The inverse of a quaternion q=(a,b,c,d)

$$q^{-1} = (a, -b, -c, -d)$$

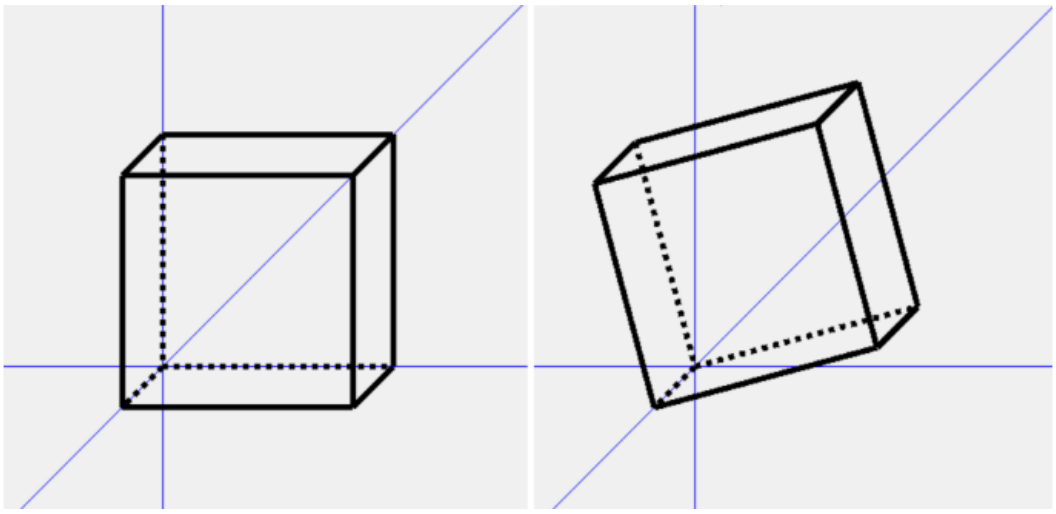
Procedure for rotating a polyhedron:

- Determine the rotation quaternion $q(q0,q1,q2,q3)$. $q0=\cos\theta/2$, $q1=x\sin\theta/2$, $q2=y\sin\theta/2$, $q3=z\sin\theta/2$, where (x,y,z) are the coordinates of the normalized direction vector of the axis around which we will rotate the polyhedron, and θ is the angle by which we will rotate around that axis (in the positive mathematical direction). Find the inverse of the rotation quaternion $q-1$.
- Convert each vertex of the polyhedron into a quaternion $pi=(0,xi,yi,zi)$, where (xi,yi,zi) are the coordinates of the i-th vertex of the polyhedron.
- For each vertex, perform the multiplication: $pi'=q-1piq$.
- Extract the complex components from the quaternion pi' as the x,y, and z coordinates of the rotated vertex.

Below is the Quaternion structure with methods for multiplication and inverse, as well as the Rotate method, which returns the rotated polyhedron $R=Polyhedron.Rotate(P,a,x,y,z)$, where (a,x,y,z) are the components of the rotation quaternion.

```
class Quaternion {
    public float a, b, c, d;
    public Quaternion(float ca, float cb, float cc, float cd) {
        a = ca; b = cb; c = cc; d = cd;
    }
    public static Quaternion Inverse(Quaternion Q) {
        return new Quaternion(Q.a, -Q.b, -Q.c, -Q.d);
    }
    public static Quaternion Multiplicate(Quaternion r, Quaternion s) {
        float t0 = r.a * s.a - r.b * s.b - r.c * s.c - r.d * s.d;
        float t1 = r.a * s.b + r.b * s.a - r.c * s.d + r.d * s.c;
        float t2 = r.a * s.c + r.c * s.a + r.b * s.d - r.d * s.b;
        float t3 = r.a * s.d + r.d * s.a - r.b * s.c + r.c * s.b;
        return new Quaternion(t0, t1, t2, t3);
    }
}

public static Polyhedron Rotate(Polyhedron P, float a, float x, float y, float z) {
    Polyhedron R = Polyhedron.Copy(P);
    Quaternion q = new Quaternion(a, x, y, z);
    Quaternion qi = Quaternion.Inverse(q);
    for (int i = 0; i < R.vertices.Length; i++) {
        float X = R.vertices[i].x;
        float Y = R.vertices[i].y;
        float Z = R.vertices[i].z;
        Quaternion p = new Quaternion(0, X, Y, Z);
        Quaternion r = Quaternion.Multiplicate(qi, p);
        r = Quaternion.Multiplicate(r, q);
        R.vertices[i].x = r.b;
        R.vertices[i].y = r.c;
        R.vertices[i].z = r.d;
    }
    return R;
}
```



7 Conclusion

Now that all parts of the code for drawing a convex polyhedron have been explained, it is time to show its result as a conclusion to this work. Below is an image of a dodecahedron (a polyhedron with 12 faces), a textual file in the correct format, and the result that the program gives in different projections.

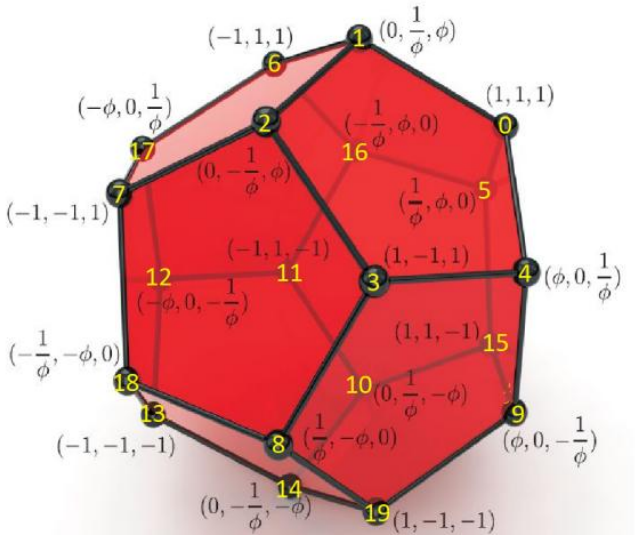
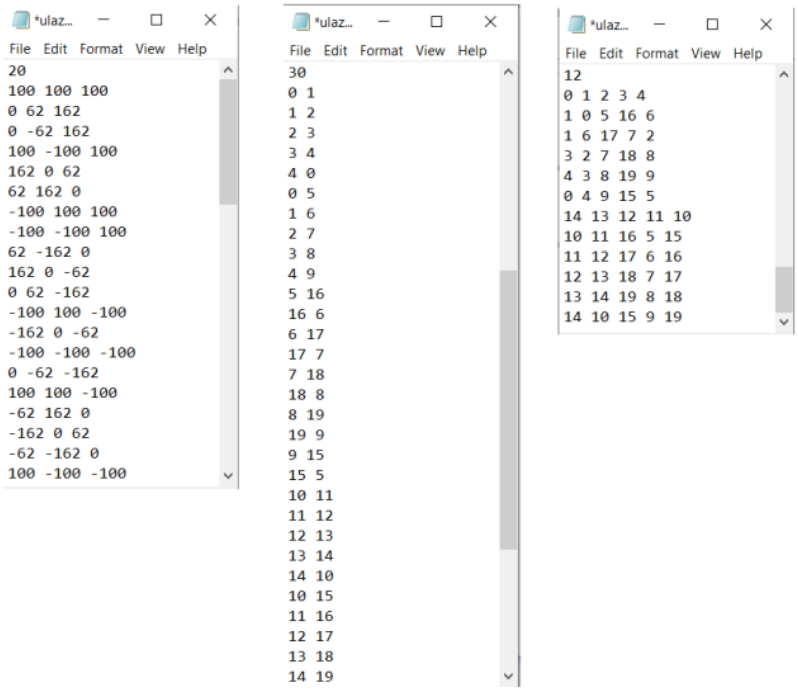


Image of a dodecahedron



*ulaz...

File Edit Format View Help

```
20
100 100 100
0 62 162
0 -62 162
100 -100 100
162 0 62
62 162 0
-100 100 100
-100 -100 100
62 -162 0
162 0 -62
0 62 -162
-100 100 -100
-162 0 -62
-100 -100 -100
0 -62 -162
100 100 -100
-62 162 0
-162 0 62
-62 -162 0
100 -100 -100
```

*ulaz...

File Edit Format View Help

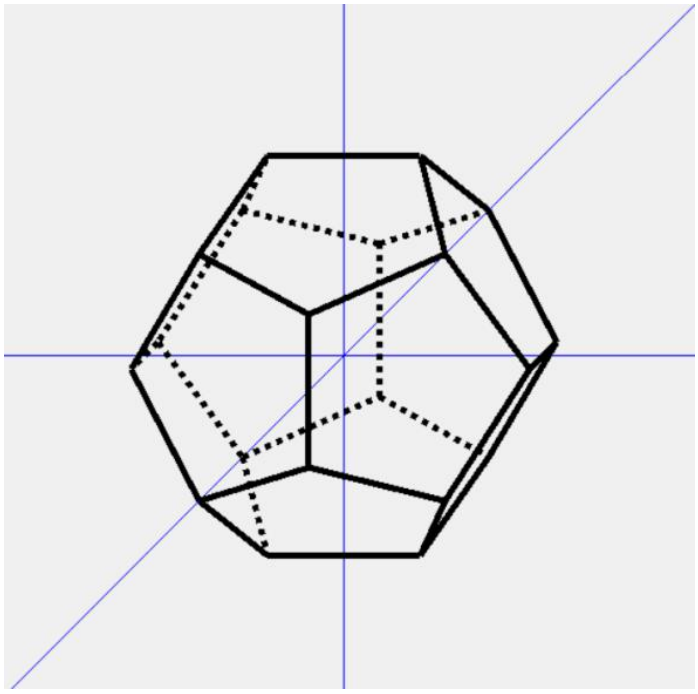
```
30
0 1
1 2
2 3
3 4
4 0
0 5
1 6
2 7
3 8
4 9
5 16
16 6
6 17
17 7
7 18
18 8
8 19
19 9
9 15
15 5
10 11
11 12
12 13
13 14
14 10
10 15
11 16
12 17
13 18
14 19
```

*ulaz...

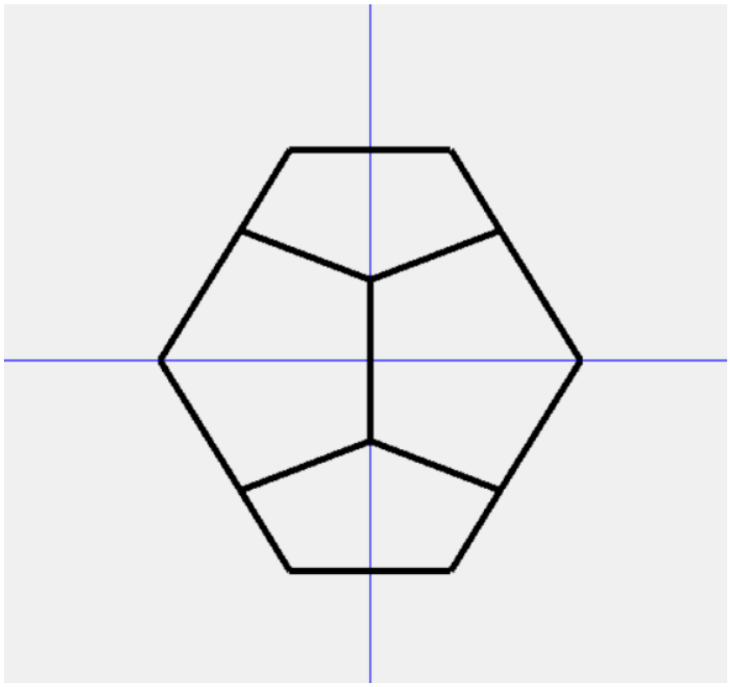
File Edit Format View Help

```
12
0 1 2 3 4
1 0 5 16 6
1 6 17 7 2
3 2 7 18 8
4 3 8 19 9
0 4 9 15 5
14 13 12 11 10
10 11 16 5 15
11 12 17 6 16
12 13 18 7 17
13 14 19 8 18
14 10 15 9 19
```

Textual file: vertices, edges, and faces



Projection at an angle of arctg4



Normal projection

References

[1] John Hughes, Andries van Dam, Morgan McGuire, David Sklar, James Foley, Steven Feiner, Kurt Akeley: *Computer Graphics - Principles and Practice* 3rd Edition, Addison-Wesley Professional; 3rd edition 2013

[2] Donald Hearn, Pauline Baker, Warren Carithers: *Computer Graphics with Open GL*, Pearson; 4th edition 2010